

Loss Functions in Neural Networks

1 Introduction

A **loss function** (also called a *cost function* or *objective function*) quantifies how well a neural network's predictions match the true labels or targets. It provides a scalar value that represents the model's performance, which is then minimized through optimization algorithms such as gradient descent.

Formally, given a dataset $\{(x_i, y_i)\}_{i=1}^N$ and a neural network $f_\theta(x)$ parameterized by θ , the loss function is:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i),$$

where $\ell(\cdot, \cdot)$ is the *sample-wise loss* measuring the discrepancy between the predicted and true outputs.

2 Key Properties

A good loss function should:

- Be **differentiable** with respect to model parameters θ (so gradients can be computed).
- Be **bounded below** to ensure convergence.
- Capture the notion of **error magnitude** — larger errors yield higher loss.
- Reflect the **task objective**, e.g., classification accuracy or reconstruction quality.

3 Regression Losses

In regression tasks, the goal is to predict continuous values.

Mean Squared Error (MSE):

$$\ell_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|f_\theta(x_i) - y_i\|_2^2.$$

This penalizes large deviations quadratically, promoting smoother fits and differentiability.

Mean Absolute Error (MAE):

$$\ell_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |f_{\theta}(x_i) - y_i|.$$

MAE is more robust to outliers, as errors grow linearly rather than quadratically.

4 Cross-Entropy Loss

Cross-entropy measures the dissimilarity between two probability distributions — the true labels y_i and the predicted probabilities $p_i = f_{\theta}(x_i)$. It is the most widely used loss in classification problems.

4.1 Binary Cross-Entropy (Logistic Loss)

For binary classification, where $y_i \in \{0, 1\}$ and $p_i = \sigma(z_i)$ is the sigmoid output,

$$\ell_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)].$$

This loss penalizes confident but incorrect predictions heavily. It is equivalent to the negative log-likelihood of a Bernoulli distribution:

$$\ell_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N \log P(y_i | x_i; \theta).$$

4.2 Categorical Cross-Entropy

For K -class classification, with one-hot encoded labels $y_{i,k} \in \{0, 1\}$ and softmax outputs $p_{i,k} = \frac{e^{z_{i,k}}}{\sum_{j=1}^K e^{z_{i,j}}}$, the cross-entropy loss is:

$$\ell_{\text{CCE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log(p_{i,k}).$$

Because only one $y_{i,k}$ equals 1 for each sample, this simplifies to:

$$\ell_{\text{CCE}} = -\frac{1}{N} \sum_{i=1}^N \log(p_{i,y_i}),$$

which corresponds to the negative log-probability of the correct class.

4.3 Probabilistic Interpretation

Cross-entropy loss arises naturally from **maximum likelihood estimation (MLE)**. If the model defines a conditional distribution $P(y|x;\theta)$, minimizing cross-entropy is equivalent to

maximizing the likelihood:

$$\min_{\theta} \ell_{\text{CCE}} \iff \max_{\theta} \sum_{i=1}^N \log P(y_i | x_i; \theta).$$

4.4 Information-Theoretic View

If $q(y|x)$ is the true label distribution (often a one-hot vector) and $p_{\theta}(y|x)$ is the model's predicted distribution, then

$$H(q, p_{\theta}) = -\mathbb{E}_{y \sim q}[\log p_{\theta}(y|x)]$$

is the **cross-entropy**. It decomposes as:

$$H(q, p_{\theta}) = H(q) + D_{\text{KL}}(q \| p_{\theta}),$$

where $H(q)$ is the true entropy (constant w.r.t. θ) and D_{KL} is the Kullback–Leibler divergence. Hence, minimizing cross-entropy is equivalent to minimizing the KL divergence between the true and predicted distributions.

4.5 Geometric Behavior

Cross-entropy produces large gradients when predictions are very wrong (e.g., $p_i \rightarrow 0$ for the correct class), helping the model learn quickly from mistakes. When predictions are confident and correct ($p_i \rightarrow 1$), the gradient diminishes, stabilizing convergence.

4.6 Summary of Cross-Entropy Properties

- Encourages accurate and confident predictions.
- Derived from probabilistic principles (MLE).
- Equivalent to minimizing KL divergence between true and predicted distributions.
- Provides stable and interpretable gradients.

5 Other Losses

Hinge Loss (used in SVM-like networks):

$$\ell_{\text{hinge}} = \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i f_{\theta}(x_i)).$$

It enforces a margin between classes and is common in linear classifiers.

Kullback–Leibler (KL) Divergence: For probabilistic outputs $p(x)$ (predicted) and $q(x)$ (true),

$$D_{\text{KL}}(q \| p) = \sum_x q(x) \log \frac{q(x)}{p(x)}.$$

KL divergence is used in variational autoencoders and knowledge distillation.

6 Regularized Loss Functions

To prevent overfitting, a regularization term $R(\theta)$ is often added:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}(\theta) + \lambda R(\theta),$$

where λ controls the strength of the penalty.

Common examples:

- $R(\theta) = \|\theta\|_2^2$ (**L2 regularization** or weight decay)
- $R(\theta) = \|\theta\|_1$ (**L1 regularization** promoting sparsity)

7 Loss Surfaces and Optimization Landscape

The **loss surface** refers to the high-dimensional landscape of $\mathcal{L}(\theta)$ as a function of model parameters. Understanding its curvature (via the Hessian) helps explain convergence behavior and generalization. In deep networks, loss surfaces are typically non-convex but contain many good local minima with similar generalization performance.

8 Conclusion

The loss function is the mathematical foundation of learning in neural networks. It defines what it means for a model to perform well, drives gradient-based optimization, and encodes the desired trade-off between accuracy, stability, and generalization. Selecting an appropriate loss function aligned with the learning objective is one of the most crucial steps in neural network design.